Printer TreeKiller

Spruce version 11.0 -- spooler version 11.0

File: DSK: KMP; ALGOL INTRO

Creation date: 27 OCT 1980 0111

For: KMP

5 total sheets = 4 pages, 1 copy.

---------- USING THE ALGOL INTERPRETER ----------

Using algol

1. Create a file of Algol code. Call it somethingorother.ALG

   Use Emacs to create file of algol code. Your file should begin with the word "begin" and "end", like all good Algol code. It should have comments describing what it does. It should be nicely indented in a format like the one shown in our example later. Some notes on things specific to this dialect of Algol:

   a. COMMENTs -- They begin with the keyword "comment" or an exclamation point ("!") and end with a semicolon.

   b. OUTPUT -- The following output commands are useful:

```
print(var);          prints the contents of var on the terminal.

print(i,n);          prints integer variable i using n columns.

print(x,n,m);        prints real variable x with n columns
                        before the decimal point and m digits after.

write("...stuff..."); types "...stuff..." literally
                        on terminal.

tab;                 outputs a tab on the terminal (like typing
                        enough spaces to move you forward to the
                        next column which is divisible by 8).

newline;             types a carriage return on the console.

space;               types a space on the console.

space(n);            types n spaces on the console.
```

2. Use the EXECUTE command at the Twenex monitor to run the program.
   eg,

```
@execute programname.alg
```

   This will create a file called programname.rel and then run it.
   If you type "ex{esc}", it will prompt with "(from)" and await a filename. This will appear as:

```
@exECUTE (FROM) programname.alg
```

3. If you want to run your Algol program again (and haven't made any changes), you can just use 'execute' on the rel file. eg,

```
@execute (FROM) programname.rel
```

   In this case, algol won't get called to create the rel file. It will just be run directly.

---------- GETTING OUTPUT TO TURN IN ----------

1. Enter PHOTO from the monitor.

   This will 'take a picture' of a console session and write it to a file. Photo will ask for a filename to write output to. Eg,

```
@PHOTO

Welcome to  PHOTO.  To exit  type 'POP' when at Exec command level.
For more help type 'HELP PHOTO' to the Exec.

Log File:
```

   When it asks for a log file, type something like "PHOTO.OUTPUT" as a filename. End your input with a {cr}. If you want help, this is *not* the time to ask for it! It wants a filename. Do *not* specify your Algol source file name here! Specify the name of a file you want PHOTO's ''picture'' to end up in.

   After accepting your filename, it will leave you in an inferior exec. During your session with that exec, everything that comes out on your terminal will show up in the file you specified. See #3 for how to leave this inferior exec to return to your normal exec. (Note that ''inferior'' here means ''under'', not ''less than adequate.'')

2. Run the things you want to photograph.

   Invoke your algol program with execute as described earlier.

3. Exit photo.

   To exit PHOTO, use the POP command from the monitor. You should do this as soon as you have successfully done all you intended to output on the line printer or as soon as you know that you didn't get the script you wanted. To use POP, you must be at the [inferior] monitor.

4. Submit the output from your photo session to the line printer.

   Use the exec print command. You probably want to delete the photo files when you are done with them as they can take up lots of disk space if everyone keeps lots of copies.

-*-*-*-*-*-*-*-*-*-*-*-*-*-*-[What NOT to Do]-*-*-*-*-*-*-*-*-*-*-*-*-*-*-

(1) Do *not* stay in PHOTO longer than just to do the problem you wanted to do. Otherwise, PHOTO will keep on recording everything you are doing.

(2) Do *not* invoke Emacs from inside PHOTO. Emacs is a display program. The line printer (where you will want to print your output) is not a display device. Needless to say, there is not a well-defined mapping between what appears on a display terminal and what goes out to the line printer if you later submit a photo of it. More precisely, what comes out on the line printer is called, in computer lingo, ''garbage.''

(3) Do *not* invoke PHOTO from inside a PHOTO unless you want to see a picture of yourself taking a picture.

A SAMPLE SESSION

Note: The Algol program shown here does nothing very interesting and it does it very inefficiently! You are right in suggesting that the real array isn't even needed to do the task it does. We provide this example just to give you a feel for how various aspects of an Algol program are supposed to look.

When Photo says it has terminated its input, you are back at your normal exec monitor and can submit the output to the line printer with the PRINT command as shown here.

Note that if "-*- Algol -*-" occurs on line 1 of your program (in a comment, of course) Emacs will automatically put you in Algol mode for editing the code when you read the file in using ^X^V.

Here's a sample console session ...

```
@PHOTO

Welcome to  PHOTO.  To exit  type 'POP' when at Exec command level.
For more help type 'HELP PHOTO' to the Exec.

Log File: PHOTO.OUTPUT

[PHOTO:  Recording initiated  Wed 22-Oct-80 3:07PM]

Link from LS.YOURNAME, TTY112
TOPS-20 Command processor 4(621)

@TYpe (file) TIMES.ALG
begin                              ! Hey, EMACS, this is -*- Algol -*- code. ;
  integer i,j;                     ! i and j are loop counters ;
  real array x[1:5,1:5];           ! x is our table of timesies ;

                                   ! Loop initializing our array ;
  for i := 1 until 5
      do for j := 1 until 5
         do x[i,j] := i*j;

  newline;                         ! Output header area ;
  write("  x");
  for i := 1 until 5 do begin tab; print(i,2) end;

                                   ! Output main table ;
  for i := 1 until 5
      do begin
           newline;
           print(i,2);
           for j:=1 until 5 do begin tab; print(x[i,j],2,1) end
         end
end
@EXecute (FROM) TIMES.ALG
LINK:   Loading
[LNKXCT TIMES execution]

  x       1       2       3       4       5
  1       1.0     2.0     3.0     4.0     5.0
  2       2.0     4.0     6.0     8.0    10.0
  3       3.0     6.0     9.0    12.0    15.0
  4       4.0     8.0    12.0    16.0    20.0
  5       5.0    10.0    15.0    20.0    25.0

End of execution.
@POP

[PHOTO:  Recording terminated  Wed 22-Oct-80 3:07PM]
@print PHOTO.OUTPUT
```